

8

Storage, Networks, and Other Peripherals

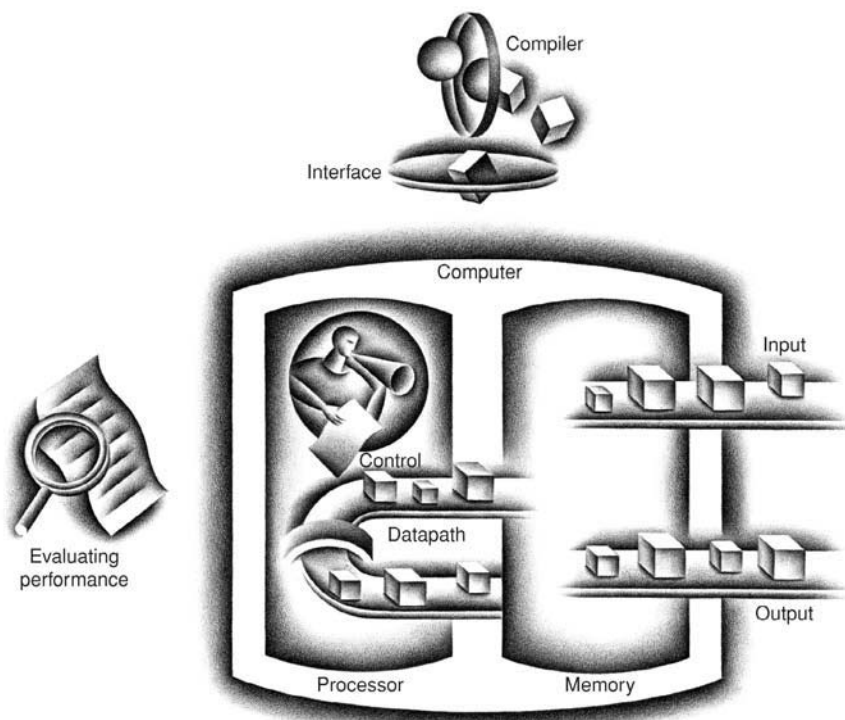
*Combining bandwidth and storage . . .
enables swift and reliable access to
the ever-expanding troves of content
on the proliferating disks and . . .
repositories of the Internet.*

George Gilder

The End Is Drawing Nigh, 2000

8.1	Introduction	566
8.2	Disk Storage and Dependability	569
 8.3	Networks	580
8.4	Buses and Other Connections between Processors, Memory, and I/O Devices	581
8.5	Interfacing I/O Devices to the Processor, Memory, and Operating System	588
8.6	I/O Performance Measures: Examples from Disk and File Systems	597
8.7	Designing an I/O System	600
8.8	Real Stuff: A Digital Camera	603
8.9	Fallacies and Pitfalls	606
8.10	Concluding Remarks	609
 8.11	Historical Perspective and Further Reading	611
8.12	Exercises	611

The Five Classic Components of a Computer



8.1 Introduction

Although users can get frustrated if their computer hangs and must be rebooted, they become apoplectic if their storage system crashes and they lose information. Thus, the bar for dependability is much higher for storage than for computation. Networks also plan for failures in communication, including several mechanisms to detect and recover from such failures. Hence, I/O systems generally place much greater emphasis on dependability and cost, while processors and memory focus on performance and cost.

I/O systems must also plan for expandability and for diversity of devices, which is not a concern for processors. Expandability is related to storage capacity, which is another design parameter for I/O systems; systems may need a lower bound of storage capacity to fulfill their role.

Although performance plays a smaller role for I/O, it is more complex. For example, with some devices we may care primarily about access latency, while with others throughput is crucial. Furthermore, performance depends on many aspects of the system: the device characteristics, the connection between the device and the rest of the system, the memory hierarchy, and the operating system. Figure 8.1 shows the structure of a simple system with its I/O. All of the components, from the individual I/O devices to the processor to the system software, will affect the dependability, expandability, and performance of tasks that include I/O.

I/O devices are incredibly diverse. Three characteristics are useful in organizing this wide variety:

- *Behavior*: Input (read once), output (write only, cannot be read), or storage (can be reread and usually rewritten).
- *Partner*: Either a human or a machine is at the other end of the I/O device, either feeding data on input or reading data on output.
- *Data rate*: The peak rate at which data can be transferred between the I/O device and the main memory or processor. It is useful to know what maximum demand the device may generate.

For example, a keyboard is an *input* device used by a *human* with a *peak data rate* of about 10 bytes per second. Figure 8.2 shows some of the I/O devices connected to computers.

In Chapter 1, we briefly discussed four important and characteristic I/O devices: mice, graphics displays, disks, and networks. In this chapter we go into much more depth on disk storage and networks.

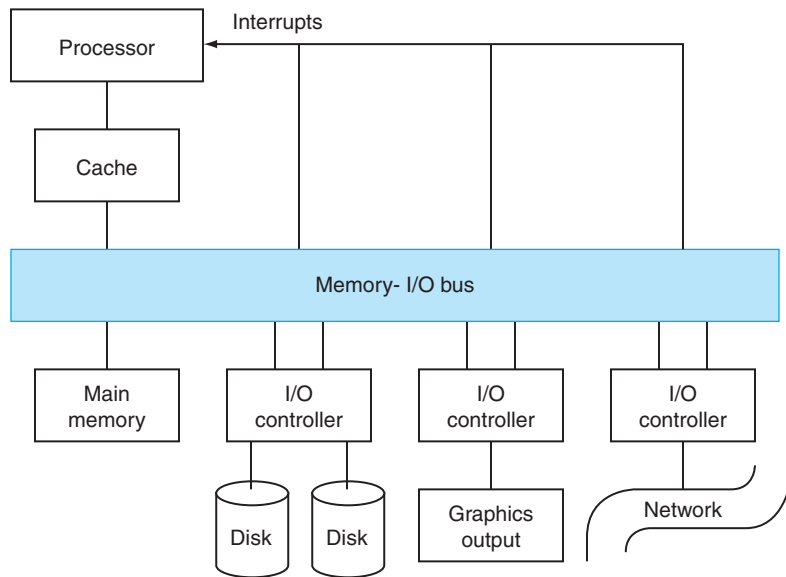


FIGURE 8.1 A typical collection of I/O devices. The connections between the I/O devices, processor, and memory are usually called *buses*. Communication among the devices and the processor use both interrupts and protocols on the bus, as we will see in this chapter. Figure 8.11 on page 585 shows the organization for a desktop PC.

How we should assess I/O performance often depends on the application. In some environments, we may care primarily about system throughput. In these cases, I/O bandwidth will be most important. Even I/O bandwidth can be measured in two different ways:

1. How much data can we move through the system in a certain time?
2. How many I/O operations can we do per unit of time?

Which performance measurement is best may depend on the environment. For example, in many multimedia applications, most I/O requests are for long streams of data, and transfer bandwidth is the important characteristic. In another environment, we may wish to process a large number of small, unrelated accesses to an I/O device. An example of such an environment might be a tax-processing office of the National Income Tax Service (NITS). NITS mostly cares about processing a large number of forms in a given time; each tax form is stored separately and is fairly small. A system oriented toward large file transfer may be satisfactory, but an I/O system that can support the simultaneous transfer of many small files may be cheaper and faster for processing millions of tax forms.

Device	Behavior	Partner	Data rate (Mbit/sec)
Keyboard	input	human	0.0001
Mouse	input	human	0.0038
Voice input	input	human	0.2640
Sound input	input	machine	3.0000
Scanner	input	human	3.2000
Voice output	output	human	0.2640
Sound output	output	human	8.0000
Laser printer	output	human	3.2000
Graphics display	output	human	800.0000–8000.0000
Modem	input or output	machine	0.0160–0.0640
Network/LAN	input or output	machine	100.0000–1000.0000
Network/wireless LAN	input or output	machine	11.0000–54.0000
Optical disk	storage	machine	80.0000
Magnetic tape	storage	machine	32.0000
Magnetic disk	storage	machine	240.0000–2560.0000

FIGURE 8.2 The diversity of I/O devices. I/O devices can be distinguished by whether they serve as input, output, or storage devices; their communication partner (people or other computers); and their peak communication rates. The data rates span eight orders of magnitude. Note that a network can be an input or an output device, but cannot be used for storage. Transfer rates for devices are always quoted in base 10, so that 10 Mbit/sec = 10,000,000 bits/sec.

I/O requests Reads or writes to I/O devices.

In other applications, we care primarily about response time, which you will recall is the total elapsed time to accomplish a particular task. If the **I/O requests** are extremely large, response time will depend heavily on bandwidth, but in many environments most accesses will be small, and the I/O system with the lowest latency per access will deliver the best response time. On single-user machines such as desktop computers and laptops, response time is the key performance characteristic.

A large number of applications, especially in the vast commercial market for computing, require both high throughput and short response times. Examples include automatic teller machines (ATMs), order entry and inventory tracking systems, file servers, and Web servers. In such environments, we care about both how long each task takes *and* how many tasks we can process in a second. The number of ATM requests you can process per hour doesn’t matter if each one takes 15 minutes—you won’t have any customers left! Similarly, if you can process each ATM request quickly but can only handle a small number of requests at once, you won’t be able to support many ATMs, or the cost of the computer per ATM will be very high.

In summary, the three classes of desktop, server, and embedded computers are sensitive to I/O dependability and cost. Desktop and embedded systems are more

focused on response time and diversity of I/O devices, while server systems are more focused on throughput and expandability of I/O devices.

8.2 Disk Storage and Dependability

As mentioned in Chapter 1, magnetic disks rely on a rotating platter coated with a magnetic surface and use a moveable read/write head to access the disk. Disk storage is **nonvolatile**—the data remains even when power is removed. A magnetic disk consists of a collection of platters (1–4), each of which has two recordable disk surfaces. The stack of platters is rotated at 5400 to 15,000 RPM and has a diameter from an inch to just over 3.5 inches. Each disk surface is divided into concentric circles, called **tracks**. There are typically 10,000 to 50,000 tracks per surface. Each track is in turn divided into **sectors** that contain the information; each track may have 100 to 500 sectors. Sectors are typically 512 bytes in size, although there is an initiative to increase the sector size to 4096 bytes. The sequence recorded on the magnetic media is a sector number, a gap, the information for that sector including error correction code (see [Appendix B](#), page B-64), a gap, the sector number of the next sector, and so on. Originally, all tracks had the same number of sectors and hence the same number of bits, but with the introduction of zone bit recording (ZBR) in the early 1990s, disk drives changed to a varying number of sectors (and hence bits) per track, instead keeping the spacing between bits constant. ZBR increases the number of bits on the outer tracks and thus increases the drive capacity.

As we saw in Chapter 1, to read and write information the read/write heads must be moved so that they are over the correct location. The disk heads for each surface are connected together and move in conjunction, so that every head is over the same track of every surface. The term *cylinder* is used to refer to all the tracks under the heads at a given point on all surfaces.

To access data, the operating system must direct the disk through a three-stage process. The first step is to position the head over the proper track. This operation is called a **seek**, and the time to move the head to the desired track is called the *seek time*.

Disk manufacturers report minimum seek time, maximum seek time, and average seek time in their manuals. The first two are easy to measure, but the average is open to wide interpretation because it depends on the seek distance. The industry has decided to calculate average seek time as the sum of the time for all possible seeks divided by the number of possible seeks. Average seek times are usually advertised as 3 ms to 14 ms, but, depending on the application and scheduling of disk requests, the actual average seek time may be only 25% to 33% of the

nonvolatile Storage device where data retains its value even when power is removed.

track One of thousands of concentric circles that makes up the surface of a magnetic disk.

sector One of the segments that make up a track on a magnetic disk; a sector is the smallest amount of information that is read or written on a disk.

seek The process of positioning a read/write head over the proper track on a disk.

rotation latency Also called delay. The time required for the desired sector of a disk to rotate under the read/write head; usually assumed to be half the rotation time.

advertised number because of locality of disk references. This locality arises both because of successive accesses to the same file and because the operating system tries to schedule such accesses together.

Once the head has reached the correct track, we must wait for the desired sector to rotate under the read/write head. This time is called the **rotational latency** or **rotational delay**. The average latency to the desired information is halfway around the disk. Because the disks rotate at 5400 RPM to 15,000 RPM, the average rotational latency is between

$$\begin{aligned}\text{Average rotational latency} &= \frac{0.5 \text{ rotation}}{5400 \text{ RPM}} = \frac{0.5 \text{ rotation}}{5400 \text{ RPM} \times \frac{60 \text{ seconds}}{1 \text{ minute}}} \\ &= 0.0056 \text{ seconds} = 5.6 \text{ ms}\end{aligned}$$

and

$$\begin{aligned}\text{Average rotational latency} &= \frac{0.5 \text{ rotation}}{15,000 \text{ RPM}} = \frac{0.5 \text{ rotation}}{15,000 \text{ RPM} \times \frac{60 \text{ seconds}}{1 \text{ minute}}} \\ &= 0.0020 \text{ seconds} = 2.0 \text{ ms}\end{aligned}$$

The last component of a disk access, *transfer time*, is the time to transfer a block of bits. The transfer time is a function of the sector size, the rotation speed, and the recording density of a track. Transfer rates in 2004 are between 30 and 80 MB/sec. The one complication is that most disk controllers have a built-in cache that stores sectors as they are passed over; transfer rates from the cache are typically higher and may be up to 320 MB/sec in 2004. Today, most disk transfers are multiple sectors in length.

A *disk controller* usually handles the detailed control of the disk and the transfer between the disk and the memory. The controller adds the final component of disk access time, *controller time*, which is the overhead the controller imposes in performing an I/O access. The average time to perform an I/O operation will consist of these four times plus any wait time incurred because other processes are using the disk.

Disk Read Time

What is the average time to read or write a 512-byte sector for a typical disk rotating at 10,000 RPM? The advertised average seek time is 6 ms, the transfer rate is 50 MB/sec, and the controller overhead is 0.2 ms. Assume that the disk is idle so that there is no waiting time.

EXAMPLE

Average disk access time is equal to Average seek time + Average rotational delay + Transfer time + Controller overhead. Using the advertised average seek time, the answer is

$$6.0 \text{ ms} + \frac{0.5 \text{ rotation}}{10,000 \text{ RPM}} + \frac{0.5 \text{ KB}}{50 \text{ MB/sec}} + 0.2 \text{ ms} = 6.0 + 3.0 + 0.01 + 0.2 = 9.2 \text{ ms}$$

If the measured average seek time is 25% of the advertised average time, the answer is

$$1.5 \text{ ms} + 3.0 \text{ ms} + 0.01 \text{ ms} + 0.2 \text{ ms} = 4.7 \text{ ms}$$

Notice that when we consider measured average seek time, as opposed to advertised average seek time, the rotational latency can be the largest component of the access time.

ANSWER

Disk densities have continued to increase for more than 50 years. The impact of this compounded improvement in density and the reduction in physical size of a disk drive has been amazing, as Figure 8.3 shows. The aims of different disk designers have led to a wide variety of drives being available at any particular time. Figure 8.4 shows the characteristics of three magnetic disks. In 2004, these disks from a single manufacturer cost between \$0.50 and \$5 per gigabyte, depending on size, interface, and performance. The smaller drive has advantages in power and volume per byte.

Elaboration: Most disk controllers include caches. Such caches allow for fast access to data that was recently read between transfers requested by the CPU. They use write through and do not update on a write miss. They often also include prefetch algorithms to try to anticipate demand. Of course, such capabilities complicate the measurement of disk performance and increase the importance of workload choice.

Dependability, Reliability, and Availability

Users crave dependable storage, but how do you define it? In the computer industry, it is harder than looking it up in the dictionary. After considerable debate, the following is considered the standard definition (Laprie 1985):

Computer system dependability is the quality of delivered service such that reliance can justifiably be placed on this service. The service delivered by a system is its observed actual behavior as perceived by other system(s) interacting with this system's users. Each module also has an ideal specified behavior, where a service specification is an agreed description of the expected behavior. A system failure occurs when the actual behavior deviates from the specified behavior.

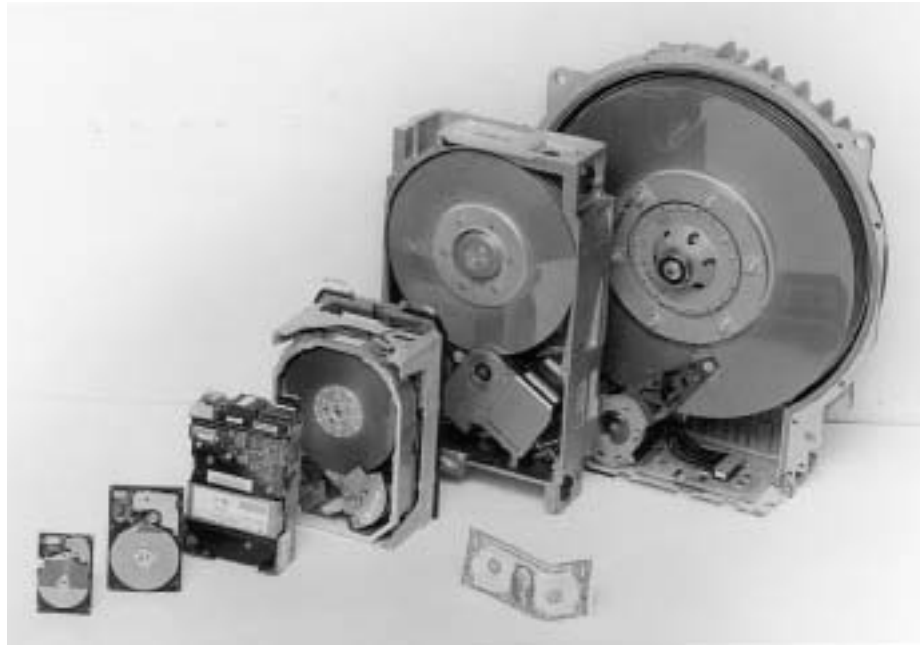


FIGURE 8.3 Six magnetic disks, varying in diameter from 14 inches down to 1.8 inches.

The IBM microdrive, not shown, has a 1-inch diameter. The pictured disks were introduced over more than 15 years ago and hence are not intended to be representative of the best capacity of modern disks of these diameters. This photograph does, however, accurately portray their relative physical sizes. The widest disk is the DEC R81, containing four 14-inch diameter platters and storing 456 MB. It was manufactured in 1985. The 8-inch diameter disk comes from Fujitsu, and this 1984 disk stores 130 MB on six platters. The Micropolis RD53 has five 5.25-inch platters and stores 85 MB. The IBM 0361 also has five platters, but these are just 3.5 inches in diameter. This 1988 disk holds 320 MB. In 2004, the most dense 3.5-inch disk had 2 platters and holds 200 GB in the same space, yielding an increase in density of about 600 times! The Conner CP 2045 has two 2.5-inch platters containing 40 MB and was made in 1990. The smallest disk in this photograph is the Integral 1820. This single 1.8-inch platter contains 20 MB and was made in 1992. Figure 8.11 on page 585 shows a 10-inch drive that holds 340 MB.

Thus, you need a reference specification of expected behavior to be able to determine dependability. Users can then see a system alternating between two states of delivered service with respect to the service specification:

1. *Service accomplishment*, where the service is delivered as specified
2. *Service interruption*, where the delivered service is different from the specified service

Transitions from state 1 to state 2 are caused by failures, and transitions from state 2 to state 1 are called *restorations*. Failures can be permanent or intermittent. The latter is the more difficult case to diagnose when a system oscillates between the two states; permanent failures are much easier to diagnose. This definition leads to two related terms: reliability and availability.

Characteristics	Seagate ST373453	Seagate ST3200822	Seagate ST94811A
Disk diameter (inches)	3.50	3.50	2.50
Formatted data capacity (GB)	73.4	200.0	40.0
Number of disk surfaces (heads)	8	4	2
Rotation speed (RPM)	15,000	7200	5400
Internal disk cache size (MB)	8	8	8
External interface, bandwidth (MB/sec)	Ultra320 SCSI, 320	Serial ATA, 150	ATA, 100
Sustained transfer rate (MB/sec)	57–86	32–58	34
Minimum seek (read/write) (ms)	0.2/0.4	1.0/1.2	1.5/2.0
Average seek read/write (ms)	3.6/3.9	8.5/9.5	12.0/14.0
Mean time to failure (MTTF) (hours)	1,200,000 @ 25°C	600,000 @ 25°C	330,000 @ 25°C
Warranty (years)	5	3	—
Nonrecoverable read errors per bits read	<1 per 10 ¹⁵	< 1 per 10 ¹⁴	< 1 per 10 ¹⁴
Temperature, vibration limits (operating)	5°–55°C, 400 Hz @ 0.5 G	0°–60°C, 350 Hz @ 0.5 G	5°–55°C, 400 Hz @ 1 G
Size: dimensions (in.), weight (pounds)	1.0" ¥ 4.0" ¥ 5.8", 1.9 lbs	1.0" ¥ 4.0" ¥ 5.8", 1.4 lbs	0.4" ¥ 2.7" ¥ 3.9", 0.2 lbs
Power: operating/idle/standby (watts)	20?/12/—	12/8/1	2.4/1.0/0.4
GB/cu. in., GB/watt	3 GB/cu.in., 4 GB/W	9 GB/cu.in., 16 GB/W	10 GB/cu.in., 17 GB/W
Price in 2004, \$/GB	^a \$400, ^a \$5/GB	^a \$100, ^a \$0.5/GB	^a \$100, ^a \$2.50/GB

FIGURE 8.4 Characteristics of three magnetic disks by a single manufacturer in 2004. The disks shown here either interface to SCSI, a standard I/O bus for many systems, or ATA, a standard I/O bus for PCs. The first disk is intended for file servers, the second for desktop PCs, and the last for laptop computers. Each disk has an 8 MB cache. The transfer rate from the cache is 3–6 times faster than the transfer rate from the disk surface. The much lower cost of the ATA 3.5-inch drive is primarily due to the hypercompetitive PC market, although there are differences in performance and reliability between it and the SCSI drive. The service life for these disks is 5 years, although Seagate offers a 5-year guarantee only on the SCSI drive, with a 1-year guarantee on the other two. Note that the quoted MTTF assumes nominal power and temperature. Disk lifetimes can be much shorter if temperature and vibration are not controlled. See the link to Seagate at www.seagate.com for more information on these drives.

Reliability is a measure of the continuous service accomplishment—or, equivalently, of the time to failure—from a reference point. Hence, the mean time to failure (MTTF) of disks in Figure 8.4 is a reliability measure. Service interruption is measured as mean time to repair (MTTR). *Mean time between failures* (MTBF) is simply the sum of MTTF + MTTR. Although MTBF is widely used, MTTF is often the more appropriate term.

Availability is a measure of the service accomplishment with respect to the alternation between the two states of accomplishment and interruption. Availability is statistically quantified as

$$\text{Availability} = \frac{\text{MTTF}}{(\text{MTTF} + \text{MTTR})}$$

Note that reliability and availability are quantifiable measures, rather than just synonyms for dependability.

What is the cause of failures? Figure 8.5 summarizes many papers that have collected data on reasons for computer systems and telecommunications systems to fail. Clearly, human operators are a significant source of failures.

small computer systems interface (SCSI) A bus used as a standard for I/O devices.

Operator	Software	Hardware	System	Year data collected
42%	25%	18%	Data center (Tandem)	1985
15%	55%	14%	Data center (Tandem)	1989
18%	44%	39%	Data center (DEC VAX)	1985
50%	20%	30%	Data center (DEC VAX)	1993
50%	14%	19%	U.S. public telephone network	1996
54%	7%	30%	U.S. public telephone network	2000
60%	25%	15%	Internet services	2002

FIGURE 8.5 Summary of studies of reasons for failures. Although it is difficult to collect data to determine if operators are the cause of errors, since operators often record the reasons for failures, these studies did capture that data. There were often other categories, such as environmental reasons for outages, but they were generally small. The top two rows come from a classic paper by Jim Gray [1990], which is still widely quoted almost 20 years after the data was collected. The next two rows are from a paper by Murphy and Gent who studied causes of outages in VAX systems over time (“Measuring system and software reliability using an automated data collection process,” *Quality and Reliability Engineering International* 11:5, September–October 1995, 341–53). The fifth and sixth rows are studies of FCC failure data about the U.S. public switched telephone network by Kuhn (“Sources of failure in the public switched telephone network,” *IEEE Computer* 30:4, April 1997, 31–36) and by Patty Enriquez. The most recent study of three Internet services is from Oppenheimer, Ganapath, and Patterson [2003].

To increase MTTF, you can improve the quality of the components or design systems to continue operation in the presence of components that have failed. Hence, failure needs to be defined with respect to a context. A failure in a component may not lead to a failure of the system. To make this distinction clear, the term *fault* is used to mean failure of a component. Here are three ways to improve MTTF:

- 1. *Fault avoidance:* preventing fault occurrence by construction
- 2. *Fault tolerance:* using redundancy to allow the service to comply with the service specification despite faults occurring, which applies primarily to hardware faults
- 3. *Fault forecasting:* predicting the presence and creation of faults, which applies to hardware and software faults

Shrinking MTTR can help availability as much as increasing MTTF. For example, tools for fault detection, diagnosis, and repair can help reduce the time to repair faults by people, software, and hardware.

redundant arrays of inexpensive disks (RAID) An organization of disks that uses an array of small and inexpensive disks so as to increase both performance and reliability.

RAID

Leveraging redundancy to improve the availability of disk storage is captured in the phrase **Redundant Arrays of Inexpensive Disks**, abbreviated **RAID**. At the time the term was coined, the alternative was large, expensive disks, such as the larger ones in Figure 8.3. The argument was that by replacing a few large disks

with many small disks, performance would improve because there would be more read heads, and there would be advantages in cost, power, and floor space since smaller disks are much more efficient per gigabyte than larger disks. Redundancy was needed because the many more smaller disks had lower reliability than a few large disks.

By having many small disks, the cost of extra redundancy to improve dependability is small relative to the large disks. Thus, dependability was more affordable if you constructed a redundant array of inexpensive disks. In retrospect, this was the key advantage.

How much redundancy do you need? Do you need extra information to find the faults? Does it matter how you organize the data and the extra check information on these disks? The paper that coined the term gave an evolutionary answer to these questions, starting with the simplest but most expensive solution. Figure 8.6 shows the evolution and example cost in number of extra check disks. To keep track of the evolution, the authors numbered the stages of RAID, and they are still used today.

No Redundancy (RAID 0)

Simply spreading data over multiple disks, called **striping**, automatically forces accesses to several disks. Striping across a set of disks makes the collection appear to software as a single large disk, which simplifies storage management. It also improves performance for large accesses, since many disks can operate at once. Video-editing systems, for example, often stripe their data and may not worry about dependability as much as, say, databases.

striping Allocation of logically sequential blocks to separate disks to allow higher performance than a single disk can deliver.

RAID 0 is something of a misnomer as there is no redundancy. However, RAID levels are often left to the operator to set when creating a storage system, and RAID 0 is often listed as one of the options. Hence, the term *RAID 0* has become widely used.

Mirroring (RAID 1)

This traditional scheme for tolerating disk failure, called **mirroring** or *shadowing*, uses twice as many disks as does RAID 0. Whenever data are written to one disk, those data are also written to a redundant disk, so that there are always two copies of the information. If a disk fails, the system just goes to the “mirror” and reads its contents to get the desired information. Mirroring is the most expensive RAID solution, since it requires the most disks.

mirroring Writing the identical data to multiple disks to increase data availability.

Error Detecting and Correcting Code (RAID 2)

RAID 2 borrows an error detection and correction scheme most often used for memories (see  **Appendix B**). Since RAID 2 has fallen into disuse, we’ll not describe it here.

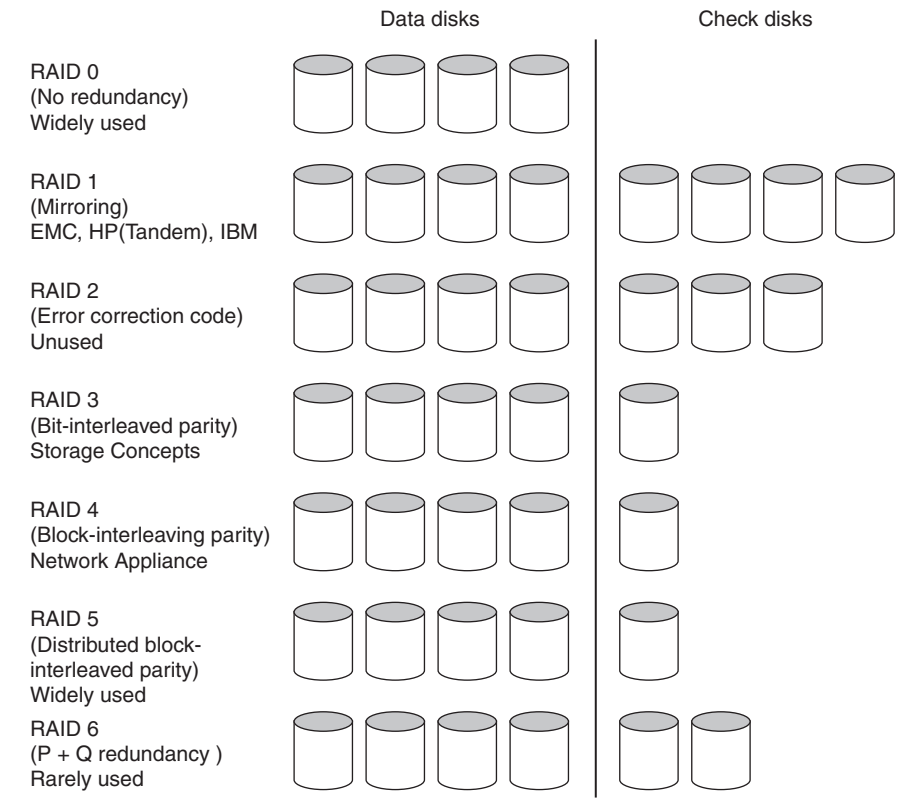


FIGURE 8.6 RAID for an example of four data disks showing extra check disks per RAID level and companies that use each level. Figures 8.7 and 8.8 explain the difference between RAID 3, RAID 4, and RAID 5.

Bit-Interleaved Parity (RAID 3)

The cost of higher availability can be reduced to $1/N$, where N is the number of disks in a **protection group**. Rather than have a complete copy of the original data for each disk, we need only add enough redundant information to restore the lost information on a failure. Reads or writes go to all disks in the group, with one extra disk to hold the check information in case there is a failure. RAID 3 is popular in applications with large data sets, such as multimedia and some scientific codes.

Parity is one such scheme. Readers unfamiliar with parity can think of the redundant disk as having the sum of all the data in the other disks. When a disk fails, then you subtract all the data in the good disks from the parity disk; the remaining information must be the missing information. Parity is simply the sum modulo two.

protection group The group of data disks or blocks that share a common check disk or block.

Unlike RAID 1, many disks must be read to determine the missing data. The assumption behind this technique is that taking longer to recover from failure but spending less on redundant storage is a good trade-off.

Block-Interleaved Parity (RAID 4)

RAID 4 uses the same ratio of data disks and check disks as RAID 3, but they access data differently. The parity is stored as blocks and associated with a set of data blocks.

In RAID 3, every access went to all disks. However, some applications prefer smaller accesses, allowing independent accesses to occur in parallel. That is the purpose of the RAID levels 4 to 6. Since error detection information in each sector is checked on reads to see if data are correct, such “small reads” to each disk can occur independently as long as the minimum access is one sector. In the RAID context, a small access goes to just one disk in a protection group while a large access goes to all the disks in a protection group.

Writes are another matter. It would seem that each small write would demand that all other disks be accessed to read the rest of the information needed to recalculate the new parity, as in Figure 8.7. A “small write” would require reading the old data and old parity, adding the new information, and then writing the new parity to the parity disk and the new data to the data disk.

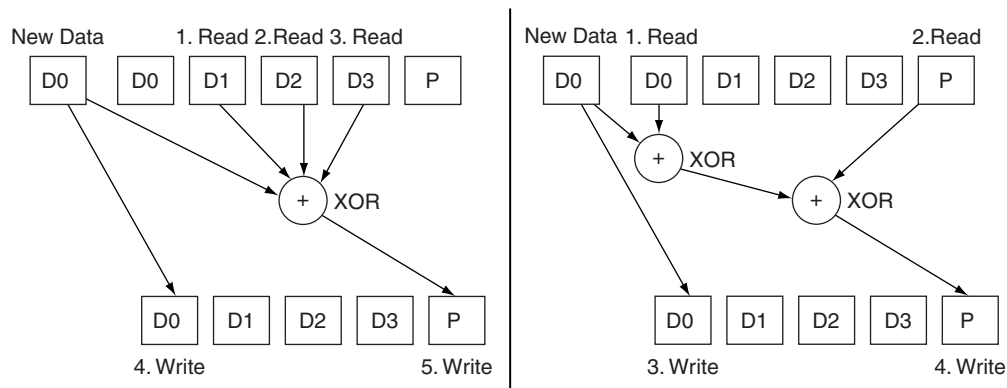


FIGURE 8.7 Small write update on RAID 3 versus RAID 4. This optimization for small writes reduces the number of disk accesses as well as the number of disks occupied. This figure assumes we have four blocks of data and one block of parity. The straightforward RAID 3 parity calculation in the left of the figure reads blocks D1, D2, and D3 before adding block D0 to calculate the new parity P (In case you were wondering, the new data D0 comes directly from the CPU, so disks are not involved in reading it.) The RAID 4 shortcut on the right reads the old value D0 and compares it to the new value D0 to see which bits will change. You then read the old parity P and then change the corresponding bits to form P. The logical function exclusive OR does exactly what we want. This example replaces three blocks reads (D1, D2, D3) and two disk writes (D0, P) involving all the disks for two disk reads (D0, P) and two disk writes (D0, P), which involve just two disks. Increasing the size of the parity group increases the savings of the shortcut. RAID 5 uses the same shortcut.

The key insight to reduce this overhead is that parity is simply a sum of information; by watching which bits change when we write the new information, we need only change the corresponding bits on the parity disk. Figure 8.7 shows the shortcut. We must read the old data from the disk being written, compare old data to the new data to see which bits change, read the old parity, change the corresponding bits, then write the new data and new parity. Thus, the small write involves four disk accesses to two disks instead of accessing all disks. This organization is RAID 4.

Distributed Block-Interleaved Parity (RAID 5)

RAID 4 efficiently supports a mixture of large reads, large writes, and small reads, plus it allows small writes. One drawback to the system is that the parity disk must be updated on every write, so the parity disk is the bottleneck for back-to-back writes.

To fix the parity-write bottleneck, the parity information can be spread throughout all the disks so that there is no single bottleneck for writes. The distributed parity organization is RAID 5.

Figure 8.8 shows how data are distributed in RAID 4 versus RAID 5. As the organization on the right shows, in RAID 5 the parity associated with each row of data blocks is no longer restricted to a single disk. This organization allows multiple writes to occur simultaneously as long as the parity blocks are not located to the same disk. For example, a write to block 8 on the right must also access its parity block P2, thereby occupying the first and third disks. A second write to block 5 on the right, implying an update to its parity block P1, accesses the second and fourth disks and thus could occur concurrently with the write to block 8. Those same writes to the organization on the left result in changes to blocks P1 and P2, both on the fifth disk, which is a bottleneck.

P + Q Redundancy (RAID 6)

Parity-based schemes protect against a single self-identifying failure. When a single failure correction is not sufficient, parity can be generalized to have a second calculation over the data and another check disk of information. This second check block allows recovery from a second failure. Thus, the storage overhead is twice that of RAID 5. The small write shortcut of Figure 8.7 works as well, except now there are six disk accesses instead of four to update both P and Q information.

RAID Summary

RAID 1 and RAID 5 are widely used in servers; one estimate is 80% of disks in servers are found in some RAID system.

One weakness of the RAID systems is repair. First, to avoid making the data unavailable during repair, the array must be designed to allow the failed disks to

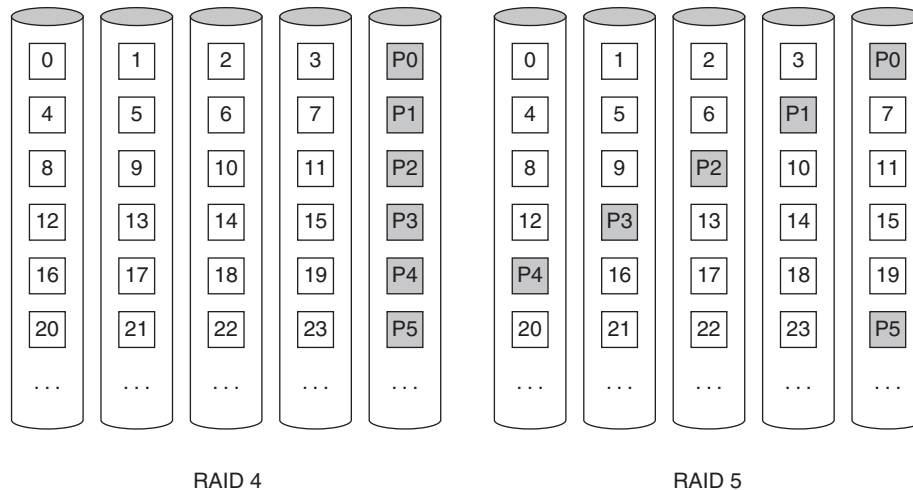


FIGURE 8.8 Block-interleaved parity (RAID 4) versus distributed block-interleaved parity (RAID 5). By distributing parity blocks to all disks, some small writes can be performed in parallel.

be replaced without having to turn off the system. RAIDs have enough redundancy to allow continuous operation, but **hot swapping** disks places demands on the physical and electrical design of the array and the disk interfaces. Second, another failure could occur during repair, so the repair time affects the chances of losing data: the longer the repair time, the greater the chances of another failure that will lose data. Rather than having to wait for the operator to bring in a good disk, some systems include **standby spares** so that the data can be reconstructed immediately upon discovery of the failure. The operator can then replace the failed disks in a more leisurely fashion. Third, although disk manufacturers quote very high MTTF for their products, those numbers are under nominal conditions. If a particular disk array has been subject to temperature cycles due to, say, the failure of the air conditioning system, or to shaking due to a poor rack design, construction, or installation, the failure rates will be much higher. The calculation of RAID reliability assumes independence between disk failures, but disk failures could be correlated because such damage due to the environment would likely happen to all the disks in the array. Finally, a human operator ultimately determines which disks to remove. As Figure 8.5 shows, operators are only human, so they occasionally remove the good disk instead of the broken disk, leading to an unrecoverable disk failure.

Although RAID 6 is rarely used today, a cautious operator might want its extra redundancy to protect against expected hardware failures plus a safety margin to protect against human error and correlated failures due to problems with the environment.

hot swapping Replacing a hardware component while the system is running.

standby spares Reserve hardware resources that can immediately take the place of a failed component.

Check Yourself

Which of the following are true about dependability?

1. If a system is up, then all its components are accomplishing their expected service.
2. Availability is a quantitative measure of the percentage of time a system is accomplishing its expected service.
3. Reliability is a quantitative measure of continuous service accomplishment by a system.
4. The major source of outages today is software.

Which of the following are true about RAID levels 1, 3, 4, 5, and 6?

1. RAID systems rely on redundancy to achieve high availability.
2. RAID 1 (mirroring) has the highest check disk overhead.
3. For small writes, RAID 3 (bit-interleaved parity) has the worst throughput.
4. For large writes, RAID 3, 4, and 5 have the same throughput.

Elaboration: One issue is how mirroring interacts with striping. Suppose you had, say, four disks worth of data to store and eight physical disks to use. Would you create four pairs of disks—each organized as RAID 1—and then stripe data across the four RAID 1 pairs? Alternatively, would you create two sets of four disks—each organized as RAID 0—and then mirror writes to both RAID 0 sets? The RAID terminology has evolved to call the former RAID 1 + 0 or RAID 10 (“striped mirrors”) and the latter RAID 0 + 1 or RAID 01 (“mirrored stripes”).



Networks

Networks are growing in popularity over time, and unlike other I/O devices, there are many books and courses on them. For readers who have not taken courses or read books on networking, Section 8.3 on the  **CD** gives a quick overview of the topics and terminology, including internetworking, the OSI model, protocol families such as TCP/IP, long-haul networks such as ATM, local area networks such as Ethernet, and wireless networks such as IEEE 802.11.

8.4

Buses and Other Connections between Processors, Memory, and I/O Devices

In a computer system, the various subsystems must have interfaces to one another. For example, the memory and processor need to communicate, as do the processor and the I/O devices. For many years, this has been done with a *bus*. A bus is a shared communication link, which uses one set of wires to connect multiple subsystems. The two major advantages of the bus organization are versatility and low cost. By defining a single connection scheme, new devices can easily be added, and peripherals can even be moved between computer systems that use the same kind of bus. Furthermore, buses are cost-effective because a single set of wires is shared in multiple ways.

The major disadvantage of a bus is that it creates a communication bottleneck, possibly limiting the maximum I/O throughput. When I/O must pass through a single bus, the bandwidth of that bus limits the maximum I/O throughput. Designing a bus system capable of meeting the demands of the processor as well as connecting large numbers of I/O devices to the machine presents a major challenge.

One reason bus design is so difficult is that the maximum bus speed is largely limited by physical factors: the length of the bus and the number of devices. These physical limits prevent us from running the bus arbitrarily fast. In addition, the need to support a range of devices with widely varying latencies and data transfer rates also makes bus design challenging.

As it becomes difficult to run many parallel wires at high speed due to clock skew and reflection, the industry is in transition from parallel shared buses to high-speed serial point-to-point interconnections with switches. Thus, such networks are gradually replacing buses in our systems.

As a result of this transition, this section has been revised in this edition to emphasize the general problem of connecting I/O devices, processors, and memory rather than focus exclusively on buses.

Bus Basics

Classically, a bus generally contains a set of control lines and a set of data lines. The control lines are used to signal requests and acknowledgments, and to indicate what type of information is on the data lines. The data lines of the bus carry information between the source and the destination. This information may consist of data, complex commands, or addresses. For example, if a disk wants to write some data into memory from a disk sector, the data lines will be used to indicate the address in memory in which to place the data as well as to carry the

bus transaction A sequence of bus operations that includes a request and may include a response, either of which may carry data. A transaction is initiated by a single request and may take many individual bus operations.

processor-memory bus A bus that connects processor and memory and that is short, generally high speed, and matched to the memory system so as to maximize memory-processor bandwidth.

backplane bus A bus that is designed to allow processors, memory, and I/O devices to coexist on a single bus.

synchronous bus A bus that includes a clock in the control lines and a fixed protocol for communicating that is relative to the clock.

asynchronous bus A bus that uses a handshaking protocol for coordinating usage rather than a clock; can accommodate a wide variety of devices of differing speeds.

actual data from the disk. The control lines will be used to indicate what type of information is contained on the data lines of the bus at each point in the transfer. Some buses have two sets of signal lines to separately communicate both data and address in a single bus transmission. In either case, the control lines are used to indicate what the bus contains and to implement the bus protocol. And because the bus is shared, we also need a protocol to decide who uses it next; we will discuss this problem shortly.

Let's consider a typical **bus transaction**. A bus transaction includes two parts: sending the address and receiving or sending the data. Bus transactions are typically defined by what they do to memory. A *read* transaction transfers data *from* memory (to either the processor or an I/O device), and a *write* transaction writes data *to* the memory. Clearly, this terminology is confusing. To avoid this, we'll try to use the terms *input* and *output*, which are always defined from the perspective of the processor: an input operation is inputting data from the device to memory, where the processor can read it, and an output operation is outputting data to a device from memory where the processor wrote it.

Busess are traditionally classified as **processor-memory buses** or *I/O buses*. Processor-memory buses are short, generally high speed, and matched to the memory system so as to maximize memory-processor bandwidth. I/O buses, by contrast, can be lengthy, can have many types of devices connected to them, and often have a wide range in the data bandwidth of the devices connected to them. I/O buses do not typically interface directly to the memory but use either a processor-memory or a **backplane bus** to connect to memory. Other buses with different characteristics have emerged for special functions, such as graphics buses.

The I/O bus serves as a way of expanding the machine and connecting new peripherals. To make this easier, the computer industry has developed several standards. The standards serve as a specification for the computer manufacturer and for the peripheral manufacturer. A standard ensures the computer designer that peripherals will be available for a new machine, and it ensures the peripheral builder that users will be able to hook up their new equipment. Figure 8.9 summarizes the key characteristics of the two dominant I/O bus standards: Firewire and USB. They connect a variety of devices to the desktop computer, from keyboards to cameras to disks.

The two basic schemes for communication on the bus are **synchronous** and **asynchronous**. If a bus is synchronous, it includes a clock in the control lines and a fixed protocol for communicating that is relative to the clock. For example, for a processor-memory bus performing a read from memory, we might have a protocol that transmits the address and read command on the first clock cycle, using the control lines to indicate the type of request. The memory might then be required to respond with the data word on the fifth clock. This type of protocol can be implemented easily in a small finite state machine. Because the protocol is predetermined and involves little logic, the bus can run very fast and the interface logic will be small.

Characteristic	Firewire (1394)	USB 2.0
Bus type	I/O	I/O
Basic data bus width (signals)	4	2
Clocking	asynchronous	asynchronous
Theoretical peak bandwidth	50 MB/sec (Firewire 400) or 100 MB/sec (Firewire 800)	0.2 MB/sec (low speed), 1.5 MB/sec (full speed), or 60 MB/sec (high speed)
Hot pluggable	yes	yes
Maximum number of devices	63	127
Maximum bus length (copper wire)	4.5 meters	5 meters
Standard name	IEEE 1394, 1394b	USE Implementors Forum

FIGURE 8.9 Key characteristics of two dominant I/O bus standards.

Synchronous buses have two major disadvantages, however. First, every device on the bus must run at the same clock rate. Second, because of clock skew problems, synchronous buses cannot be long if they are fast (see [Appendix B](#) for a discussion of clock skew). Processor-memory buses are often synchronous because the devices communicating are close, small in number, and prepared to operate at high clock rates.

An asynchronous bus is not clocked. Because it is not clocked, an asynchronous bus can accommodate a wide variety of devices, and the bus can be lengthened without worrying about clock skew or synchronization problems. Both Firewire and USE 2.0 are asynchronous. To coordinate the transmission of data between sender and receiver, an asynchronous bus uses a **handshaking protocol**. A handshaking protocol consists of a series of steps in which the sender and receiver proceed to the next step only when both parties agree. The protocol is implemented with an additional set of control lines.

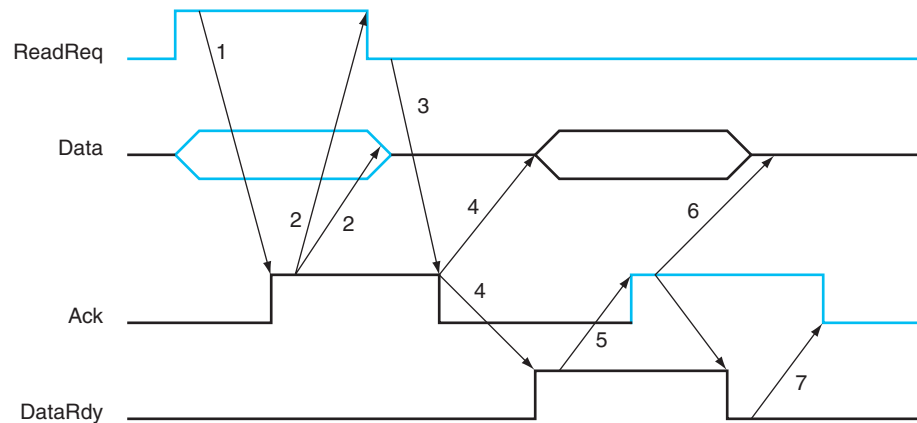
A simple example will illustrate how asynchronous buses work. Let's consider a device requesting a word of data from the memory system. Assume that there are three control lines:

1. *ReadReq*: Used to indicate a read request for memory. The address is put on the data lines at the same time.
2. *DataRdy*: Used to indicate that the data word is now ready on the data lines. In an output transaction, the memory will assert this signal since it is providing the data. In an input transaction, an I/O device would assert this signal, since it would provide data. In either case, the data is placed on the data lines at the same time.
3. *Ack*: Used to acknowledge the ReadReq or the DataRdy signal of the other party.

handshaking protocol A series of steps used to coordinate asynchronous bus transfers in which the sender and receiver proceed to the next step only when both parties agree that the current step has been completed.

In an asynchronous protocol, the control signals ReadReq and DataRdy are asserted until the other party (the memory or the device) indicates that the control lines have been seen and the data lines have been read; this indication is made by asserting the Ack line. This complete process is called *handshaking*. Figure 8.10 shows how such a protocol operates by depicting the steps in the communication.

Although much of the bandwidth of a bus is decided by the choice of a synchronous or asynchronous protocol and the timing characteristics of the bus, several other factors affect the bandwidth that can be attained by a single transfer. The most important of these are the data bus width, and whether it supports block transfers or it transfers a word at a time.



The steps in the protocol begin immediately after the device signals a request by raising ReadReq and putting the address on the Data lines:

1. When memory sees the ReadReq line, it reads the address from the data bus and raises Ack to indicate it has been seen.
2. I/O device sees the Ack line high and releases the ReadReq and data lines.
3. Memory sees that ReadReq is low and drops the Ack line to acknowledge the ReadReq signal.
4. This step starts when the memory has the data ready. It places the data from the read request on the data lines and raises DataRdy.
5. The I/O device sees DataRdy, reads the data from the bus, and signals that it has the data by raising Ack.
6. The memory sees the Ack signal, drops DataRdy, and releases the data lines.
7. Finally, the I/O device, seeing DataRdy go low, drops the Ack line, which indicates that the transmission is completed.

A new bus transaction can now begin.

FIGURE 8.10 The asynchronous handshaking protocol consists of seven steps to read a word from memory and receive it in an I/O device. The signals in color are those asserted by the I/O device, while the memory asserts the signals shown in black. The arrows label the seven steps and the event that triggers each step. The symbol showing two lines (high and low) at the same time on the data lines indicates that the data lines have valid data at this point. (The symbol indicates that the data is valid, but the value is not known.)

Elaboration: Another method for increasing the effective bus bandwidth is to release the bus when it is not being used for transmitting information. This type of protocol is called a [split transaction protocol](#). The advantage of such a protocol is that, by freeing the bus during the time data is not being transmitted, the protocol allows another requestor to use the bus. This can improve the effective bus bandwidth for the entire system if the memory is sophisticated enough to handle multiple overlapping transactions. Multiprocessors sharing a memory bus may use split transaction protocols.

split transaction protocol A protocol in which the bus is released during a bus transaction while the requester is waiting for the data to be transmitted, which frees the bus for access by another requester.

The Buses and Networks of the Pentium 4

Figure 8.11 shows the I/O system of a PC based on the Pentium 4. The processor connects to peripherals via two main chips. The chip next to the processor is the memory controller hub, commonly called the *north bridge*, and the one connected to it is the I/O controller hub, called the *south bridge*.

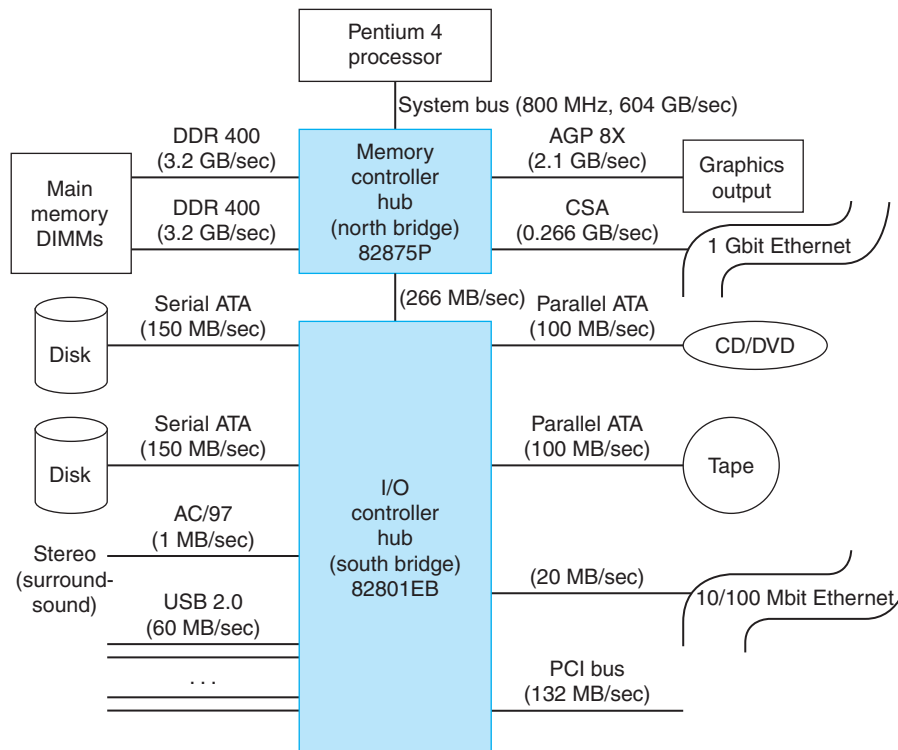


FIGURE 8.11 Organization of the I/O system on a Pentium 4 PC using the Intel 875 chip set. Note that the maximum transfer rate between the north bridge (memory hub) and south bridge (I/O hub) is 266 MB/sec, which is why Intel put the AGP bus and Gigabit Ethernet on the north bridge.

The north bridge is basically a DMA controller, connecting the processor to memory, the AGP graphic bus, and the south bridge chip. The south bridge connects the north bridge to a cornucopia of I/O buses. Intel and others offer a wide variety of these chip sets to connect the Pentium 4 to the outside world. To give a flavor of the options, Figure 8.12 shows two of the chip sets.

As Moore’s law continues, an increasing number of I/O controllers that were formerly available as optional cards that connected to I/O buses have been co-opted into these chip sets. For example, the south bridge chip of the Intel 875 includes a striping RAID controller, and the north bridge chip of the Intel 845GL includes a graphics controller.

	875P chip set	845GL chip set
Target segment	Performance PC	Value PC
System bus (64 bit)	800/533 MHz	400 MHz
Memory controller hub (“north bridge”)		
Package size, pins	42.5 ¥ 42.5 mm, 1005	37.5 ¥ 37.5 mm, 760
Memory speed	DDR 400/333/266 SDRAM	DDR 266/200, PC133 SDRAM
Memory buses, widths	2 ¥ 72	1 ¥ 64
Number of DIMMs, DRAM Mbit support	4, 128/256/512 Mbits	2, 128/256/512 MBits
Maximum memory capacity	4 GB	2 GB
Memory error correction available?	yes	no
AGP graphics bus, speed	yes, 8X or 4X	no
Graphics controller	external	Internal (Extreme Graphics)
CSA Gigabit Ethernet interface	yes	no
South bridge interface speed (8 bit)	266 MHz	266 MHz
I/O controller hub (“south bridge”)		
Package size, pins	31 ¥ 31 mm, 460	31 ¥ 31 mm, 421
PCI bus: width, speed, masters	32-bit, 33 MHz, 6 masters	32-bit, 33 MHz, 6 masters
Ethernet MAC controller, interface	100/10 Mbit	100/10 Mbit
USB 2.0 ports, controllers	8, 4	6, 3
ATA 100 ports	2	2
Serial ATA 150 controller, ports	yes, 2	no
RAID 0 controller	yes	no
AC-97 audio controller, interface	yes	yes
I/O management	SMbus 2.0, GPIO	SMbus 2.0, GPIO

FIGURE 8.12 Two Pentium 4 I/O chip sets from Intel. The 845GL north bridge uses many fewer pins than the 875 by having just one memory bus and by omitting the AGP bus and the Gigabit Ethernet interface. Note that the serial nature of USB and Serial ATA means that two more USB ports and two more Serial ATA ports need just 39 more pins in the south bridge of the 875 versus the 845GL chip sets.

These two chips demonstrate the gradual evolution from parallel shared buses to high-speed serial point-to-point interconnections with switches via the past and future versions of ATA and PCI.

Serial ATA is a serial successor to the parallel ATA bus used by magnetic and optical disks in PCs. The first generation transfers at 150 MB/sec compared to the 100 MB/sec of parallel ATA-100 bus. Its distance is 1 meter, twice the maximum length of ATA-100. It uses just 7 wires, with one 2-wire data channel in each direction, compared to 80 for ATA-100.

The south bridge in Figure 8.11 demonstrates the transitory period between parallel buses and serial networks by providing both parallel and serial ATA buses.

PCI Express is a serial successor to the popular PCI bus. Rather than 32–64 shared wires operating at 33 MHz–133 MHz with a peak bandwidth of 132–1064 MB/sec, PCI Express uses just 4 wires in each direction operating at 625 MHz to offer 300 MB/sec per direction. The bandwidth per pin of PCI Express is 5–10 times its predecessors. A computer can then afford to have several PCI Express interfaces to get even higher bandwidth.

Although the chips in Figure 8.11 only show the parallel PCI bus, Intel plans to replace the AGP graphics bus and the bus between the north bridge and the south bridge with PCI Express in the next generation of these chips.

Buses and networks provide electrical interconnection among I/O devices, processors, and memory, and also define the lowest-level protocol for communication. Above this basic level, we must define hardware and software protocols for controlling data transfers between I/O devices and memory, and for the processor to specify commands to the I/O devices. These topics are covered in the next section.

Both networks and buses connect components together. Which of the following are true about them?

1. Networks and I/O buses are almost always standardized.
2. Shared media networks and multimaster buses need an arbitration scheme.
3. Local area networks and processor-memory buses are almost always synchronous.
4. High-performance networks and buses use similar techniques compared to their lower-performance alternatives: they are wider, send many words per transaction, and have separate address and data lines.

**Check
Yourself**

8.5

Interfacing I/O Devices to the Processor, Memory, and Operating System

A bus or network protocol defines how a word or block of data should be communicated on a set of wires. This still leaves several other tasks that must be performed to actually cause data to be transferred from a device and into the memory address space of some user program. This section focuses on these tasks and will answer such questions as the following:

- How is a user I/O request transformed into a device command and communicated to the device?
- How is data actually transferred to or from a memory location?
- What is the role of the operating system?

As we will see in answering these questions, the operating system plays a major role in handling I/O, acting as the interface between the hardware and the program that requests I/O.

The responsibilities of the operating system arise from three characteristics of I/O systems:

1. Multiple programs using the processor share the I/O system.
2. I/O systems often use interrupts (externally generated exceptions) to communicate information about I/O operations. Because interrupts cause a transfer to kernel or supervisor mode, they must be handled by the operating system (OS).
3. The low-level control of an I/O device is complex because it requires managing a set of concurrent events and because the requirements for correct device control are often very detailed.

Hardware Software Interface

The three characteristics of I/O systems above lead to several different functions the OS must provide:

- The OS guarantees that a user's program accesses only the portions of an I/O device to which the user has rights. For example, the OS must not allow a program to read or write a file on disk if the owner of the file has not granted access to this program. In a system with shared I/O devices, protection could not be provided if user programs could perform I/O directly.
- The OS provides abstractions for accessing devices by supplying routines that handle low-level device operations.

- The OS handles the interrupts generated by I/O devices, just as it handles the exceptions generated by a program.
- The OS tries to provide equitable access to the shared I/O resources, as well as schedule accesses in order to enhance system throughput.

To perform these functions on behalf of user programs, the operating system must be able to communicate with the I/O devices and to prevent the user program from communicating with the I/O devices directly. Three types of communication are required:

1. The OS must be able to give commands to the I/O devices. These commands include not only operations like read and write, but also other operations to be done on the device, such as a disk seek.
2. The device must be able to notify the OS when the I/O device has completed an operation or has encountered an error. For example, when a disk completes a seek, it will notify the OS.
3. Data must be transferred between memory and an I/O device. For example, the block being read on a disk read must be moved from disk to memory.

In the next few sections, we will see how these communications are performed.

Giving Commands to I/O Devices

To give a command to an I/O device, the processor must be able to address the device and to supply one or more command words. Two methods are used to address the device: memory-mapped I/O and special I/O instructions. In **memory-mapped I/O**, portions of the address space are assigned to I/O devices. Reads and writes to those addresses are interpreted as commands to the I/O device.

For example, a write operation can be used to send data to an I/O device where the data will be interpreted as a command. When the processor places the address and data on the memory bus, the memory system ignores the operation because the address indicates a portion of the memory space used for I/O. The device controller, however, sees the operation, records the data, and transmits it to the device as a command. User programs are prevented from issuing I/O operations directly because the OS does not provide access to the address space assigned to the I/O devices and thus the addresses are protected by the address translation. Memory-mapped I/O can also be used to transmit data by writing or reading to select addresses. The device uses the address to determine the type of command, and the data may be provided by a write or obtained by a read. In any event, the address encodes both the device identity and the type of transmission between processor and device.

memory-mapped I/O An I/O scheme in which portions of address space are assigned to I/O devices and reads and writes to those addresses are interpreted as commands to the I/O device.

Actually performing a read or write of data to fulfill a program request usually requires several separate I/O operations. Furthermore, the processor may have to interrogate the status of the device between individual commands to determine whether the command completed successfully. For example, a simple printer has two I/O device registers—one for status information and one for data to be printed. The Status register contains a *done bit*, set by the printer when it has printed a character, and an *error bit*, indicating that the printer is jammed or out of paper. Each byte of data to be printed is put into the Data register. The processor must then wait until the printer sets the done bit before it can place another character in the buffer. The processor must also check the error bit to determine if a problem has occurred. Each of these operations requires a separate I/O device access.

I/O instructions A dedicated instruction that is used to give a command to an I/O device and that specifies both the device number and the command word (or the location of the command word in memory).

Elaboration: The alternative to memory-mapped I/O is to use dedicated **I/O instructions** in the processor. These I/O instructions can specify both the device number and the command word (or the location of the command word in memory). The processor communicates the device address via a set of wires normally included as part of the I/O bus. The actual command can be transmitted over the data lines in the bus. Examples of computers with I/O instructions are the Intel IA-32 and the IBM 370 computers. By making the I/O instructions illegal to execute when not in kernel or supervisor mode, user programs can be prevented from accessing the devices directly.

polling The process of periodically checking the status of an I/O device to determine the need to service the device.

Communicating with the Processor

The process of periodically checking status bits to see if it is time for the next I/O operation, as in the previous example, is called **polling**. Polling is the simplest way for an I/O device to communicate with the processor. The I/O device simply puts the information in a Status register, and the processor must come and get the information. The processor is totally in control and does all the work.

Polling can be used in several different ways. Real-time embedded applications poll the I/O devices since the I/O rates are predetermined and it makes I/O overhead more predictable, which is helpful for real time. As we will see, this allows polling to be used even when the I/O rate is somewhat higher.

The disadvantage of polling is that it can waste a lot of processor time because processors are so much faster than I/O devices. The processor may read the Status register many times, only to find that the device has not yet completed a comparatively slow I/O operation, or that the mouse has not budged since the last time it was polled. When the device completes an operation, we must still read the status to determine whether it was successful.

The overhead in a polling interface was recognized long ago, leading to the invention of interrupts to notify the processor when an I/O device requires attention from the processor. **Interrupt-driven I/O**, which is used by almost all systems

interrupt-driven I/O An I/O scheme that employs interrupts to indicate to the processor that an I/O device needs attention.

for at least some devices, employs I/O interrupts to indicate to the processor that an I/O device needs attention. When a device wants to notify the processor that it has completed some operation or needs attention, it causes the processor to be interrupted.

An I/O interrupt is just like the exceptions we saw in Chapters 5, 6, and 7, with two important exceptions:

1. An I/O interrupt is asynchronous with respect to the instruction execution. That is, the interrupt is not associated with any instruction and does not prevent the instruction completion. This is very different from either page fault exceptions or exceptions such as arithmetic overflow. Our control unit need only check for a pending I/O interrupt at the time it starts a new instruction.
2. In addition to the fact that an I/O interrupt has occurred, we would like to convey further information such as the identity of the device generating the interrupt. Furthermore, the interrupts represent devices that may have different priorities and whose interrupt requests have different urgencies associated with them.

To communicate information to the processor, such as the identity of the device raising the interrupt, a system can use either vectored interrupts or an exception Cause register. When the processor recognizes the interrupt, the device can send either the vector address or a status field to place in the Cause register. As a result, when the OS gets control, it knows the identity of the device that caused the interrupt and can immediately interrogate the device. An interrupt mechanism eliminates the need for the processor to poll the device and instead allows the processor to focus on executing programs.

Interrupt Priority Levels

To deal with the different priorities of the I/O devices, most interrupt mechanisms have several levels of priority: UNIX operating systems use four to six levels. These priorities indicate the order in which the processor should process interrupts. Both internally generated exceptions and external I/O interrupts have priorities; typically, I/O interrupts have lower priority than internal exceptions. There may be multiple I/O interrupt priorities, with high-speed devices associated with the higher priorities.

To support priority levels for interrupts, MIPS provides the primitives that let the operating system implement the policy, similar to how MIPS handles TLB misses. Figure 8.13 shows the key registers, and Section A.7 in  [Appendix A](#) gives more details.

The Status register determines who can interrupt the computer. If the interrupt enable bit is 0, then none can interrupt. A more refined blocking of interrupts is available in the interrupt mask field. There is a bit in the mask corresponding to

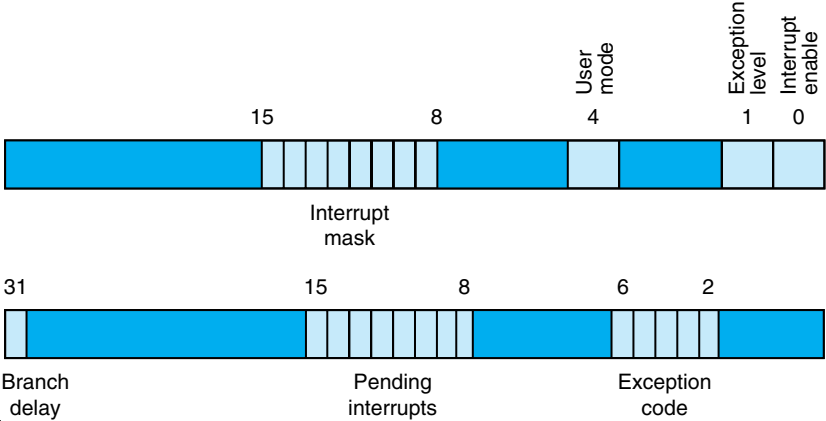


FIGURE 8.13 The Cause and Status registers. This version of the Cause register corresponds to the MIPS-32 architecture. The earlier MIPS I architecture had three nested sets of kernel/user and interrupt enable bits to support nested interrupts. Section A.7 in [Appendix A](#) has more details about these registers.

each bit in the pending interrupt field of the Cause register. To enable the corresponding interrupt, there must be a 1 in the mask field at that bit position. Once an interrupt occurs, the operating system can find the reason in the exception code field of the Status register: 0 means an interrupt occurred, with other values for the exceptions mentioned in Chapter 7.

Here are the steps that must occur in handling an interrupt:

1. Logically AND the pending interrupt field and the interrupt mask field to see which enabled interrupts could be the culprit. Copies are made of these two registers using the `mfc0` instruction.
2. Select the higher priority of these interrupts. The software convention is that the leftmost is the highest priority.
3. Save the interrupt mask field of the Status register.
4. Change the interrupt mask field to disable all interrupts of equal or lower priority.
5. Save the processor state needed to handle the interrupt.
6. To allow higher-priority interrupts, set the interrupt enable bit of the Cause register to 1.
7. Call the appropriate interrupt routine.
8. Before restoring state, set the interrupt enable bit of the Cause register to 0. This allows you to restore the interrupt mask field.

■ **Appendix A** shows an exception handler for a simple I/O task on pages A-36 to A-37.

How do the *interrupt priority levels* (IPL) correspond to these mechanisms? The IPL is an operating system invention. It is stored in the memory of the process, and every process is given an IPL. At the lowest IPL, all interrupts are permitted. Conversely, at the highest IPL, all interrupts are blocked. Raising and lowering the IPL involves changes to the interrupt mask field of the Status register.

Elaboration: The two least significant bits of the pending interrupt and interrupt mask fields are for software interrupts, which are lower priority. These are typically used by higher-priority interrupts to leave work for lower-priority interrupts to do once the immediate reason for the interrupt is handled. Once the higher-priority interrupt is finished, the lower-priority tasks will be noticed and handled.

Transferring the Data between a Device and Memory

We have seen two different methods that enable a device to communicate with the processor. These two techniques—polling and I/O interrupts—form the basis for two methods of implementing the transfer of data between the I/O device and memory. Both these techniques work best with lower-bandwidth devices, where we are more interested in reducing the cost of the device controller and interface than in providing a high-bandwidth transfer. Both polling and interrupt-driven transfers put the burden of moving data and managing the transfer on the processor. After looking at these two schemes, we will examine a scheme more suitable for higher-performance devices or collections of devices.

We can use the processor to transfer data between a device and memory based on polling. In real-time applications, the processor loads data from I/O device registers and stores them into memory.

An alternative mechanism is to make the transfer of data interrupt driven. In this case, the OS would still transfer data in small numbers of bytes from or to the device. But because the I/O operation is interrupt driven, the OS simply works on other tasks while data is being read from or written to the device. When the OS recognizes an interrupt from the device, it reads the status to check for errors. If there are none, the OS can supply the next piece of data, for example, by a sequence of memory-mapped writes. When the last byte of an I/O request has been transmitted and the I/O operation is completed, the OS can inform the program. The processor and OS do all the work in this process, accessing the device and memory for each data item transferred.

Interrupt-driven I/O relieves the processor from having to wait for every I/O event, although if we used this method for transferring data from or to a hard disk, the overhead could still be intolerable, since it could consume a large fraction of the processor when the disk was transferring. For high-bandwidth devices like hard disks, the transfers consist primarily of relatively large blocks of data

direct memory access (DMA)

A mechanism that provides a device controller the ability to transfer data directly to or from the memory without involving the processor.

bus master A unit on the bus that can initiate bus requests.

(hundreds to thousands of bytes). Thus, computer designers invented a mechanism for offloading the processor and having the device controller transfer data directly to or from the memory without involving the processor. This mechanism is called **direct memory access (DMA)**. The interrupt mechanism is still used by the device to communicate with the processor, but only on completion of the I/O transfer or when an error occurs.

DMA is implemented with a specialized controller that transfers data between an I/O device and memory independent of the processor. The DMA controller becomes the **bus master** and directs the reads or writes between itself and memory. There are three steps in a DMA transfer:

1. The processor sets up the DMA by supplying the identity of the device, the operation to perform on the device, the memory address that is the source or destination of the data to be transferred, and the number of bytes to transfer.
2. The DMA starts the operation on the device and arbitrates for the bus. When the data is available (from the device or memory), it transfers the data. The DMA device supplies the memory address for the read or the write. If the request requires more than one transfer on the bus, the DMA unit generates the next memory address and initiates the next transfer. Using this mechanism, a DMA unit can complete an entire transfer, which may be thousands of bytes in length, without bothering the processor. Many DMA controllers contain some memory to allow them to deal flexibly with delays either in transfer or those incurred while waiting to become bus master.
3. Once the DMA transfer is complete, the controller interrupts the processor, which can then determine by interrogating the DMA device or examining memory whether the entire operation completed successfully.

There may be multiple DMA devices in a computer system. For example, in a system with a single processor-memory bus and multiple I/O buses, each I/O bus controller will often contain a DMA processor that handles any transfers between a device on the I/O bus and the memory.

Unlike either polling or interrupt-driven I/O, DMA can be used to interface a hard disk without consuming all the processor cycles for a single I/O. Of course, if the processor is also contending for memory, it will be delayed when the memory is busy doing a DMA transfer. By using caches, the processor can avoid having to access memory most of the time, thereby leaving most of the memory bandwidth free for use by I/O devices.

Elaboration: To further reduce the need to interrupt the processor and occupy it in handling an I/O request that may involve doing several actual operations, the I/O controller can be made more intelligent. Intelligent controllers are often called *I/O proces-*

sors (as well as *I/O controllers* or *channel controllers*). These specialized processors basically execute a series of I/O operations, called an *I/O program*. The program may be stored in the I/O processor, or it may be stored in memory and fetched by the I/O processor. When using an I/O processor, the operating system typically sets up an I/O program that indicates the I/O operations to be done as well as the size and transfer address for any reads or writes. The I/O processor then takes the operations from the I/O program and interrupts the processor only when the entire program is completed. DMA processors are essentially special-purpose processors (usually single-chip and nonprogrammable), while I/O processors are often implemented with general-purpose microprocessors, which run a specialized I/O program.

Direct Memory Access and the Memory System

When DMA is incorporated into an I/O system, the relationship between the memory system and processor changes. Without DMA, all accesses to the memory system come from the processor and thus proceed through address translation and cache access as if the processor generated the references. With DMA, there is another path to the memory system—one that does not go through the address translation mechanism or the cache hierarchy. This difference generates some problems in both virtual memory systems and systems with caches. These problems are usually solved with a combination of hardware techniques and software support.

The difficulties in having DMA in a virtual memory system arise because pages have both a physical and a virtual address. DMA also creates problems for systems with caches because there can be two copies of a data item: one in the cache and one in memory. Because the DMA processor issues memory requests directly to the memory rather than through the processor cache, the value of a memory location seen by the DMA unit and the processor may differ. Consider a read from disk that the DMA unit places directly into memory. If some of the locations into which the DMA writes are in the cache, the processor will receive the old value when it does a read. Similarly, if the cache is write-back, the DMA may read a value directly from memory when a newer value is in the cache, and the value has not been written back. This is called the *stale data problem* or *coherence problem*.

In a system with virtual memory, should DMA work with virtual addresses or physical addresses? The obvious problem with virtual addresses is that the DMA unit will need to translate the virtual addresses to physical addresses. The major problem with the use of a physical address in a DMA transfer is that the transfer cannot easily cross a page boundary. If an I/O request crossed a page boundary, then the memory locations to which it was being transferred would not necessarily be contiguous in the virtual memory. Consequently, if we use physical addresses, we must constrain all DMA transfers to stay within one page.

One method to allow the system to initiate DMA transfers that cross page boundaries is to make the DMA work on virtual addresses. In such a system, the DMA unit has a small number of map entries that provide virtual-to-physical mapping for a transfer. The operating system provides the mapping when the I/O is initiated. By using this mapping, the DMA unit need not worry about the location of the virtual pages involved in the transfer.

Another technique is for the operating system to break the DMA transfer into a series of transfers, each confined within a single physical page. The transfers are then *chained* together and handed to an I/O processor or intelligent DMA unit that executes the entire sequence of transfers; alternatively, the operating system can individually request the transfers.

Whichever method is used, the operating system must still cooperate by not remapping pages while a DMA transfer involving that page is in progress.

We have looked at three different methods for transferring data between an I/O device and memory. In moving from polling to an interrupt-driven to a DMA interface, we shift the burden for managing an I/O operation from the processor to a progressively more intelligent I/O controller. These methods have the advantage of freeing up processor cycles. Their disadvantage is that they increase the cost of the I/O system. Because of this, a given computer system can choose which point along this spectrum is appropriate for the I/O devices connected to it.

Before discussing the design of I/O systems, let's look briefly at performance measures of them.

Hardware Software Interface

The coherency problem for I/O data is avoided by using one of three major techniques. One approach is to route the I/O activity through the cache. This ensures that reads see the latest value while writes update any data in the cache. Routing all I/O through the cache is expensive and potentially has a large negative performance impact on the processor, since the I/O data is rarely used immediately and may displace useful data that a running program needs. A second choice is to have the OS selectively invalidate the cache for an I/O read or force write-backs to occur for an I/O write (often called *cache flushing*). This approach requires some small amount of hardware support and is probably more efficient if the software can perform the function easily and efficiently. Because this flushing of large parts of the cache need only happen on DMA block accesses, it will be relatively infrequent. The third approach is to provide a hardware mechanism for selectively flushing (or invalidating) cache entries. Hardware invalidation to ensure cache coherence is typical in multiprocessor systems, and the same technique can be used for I/O; we discuss this topic in detail in Chapter 9.

In ranking of the three ways of doing I/O, which statements are true?

1. If we want the lowest latency for an I/O operation to a single I/O device, the order is polling, DMA, and interrupt driven.
2. In terms of lowest impact on processor utilization from a single I/O device, the order is DMA, interrupt driven, and polling

**Check
Yourself**

8.6

I/O Performance Measures: Examples from Disk and File Systems

How should we compare I/O systems? This is a complex question because I/O performance depends on many aspects of the system and different applications stress different aspects of the I/O system. Furthermore, a design can make complex trade-offs between response time and throughput, making it impossible to measure just one aspect in isolation. For example, handling a request as early as possible generally minimizes response time, although greater throughput can be achieved if we try to handle related requests together. Accordingly, we may increase throughput on a disk by grouping requests that access locations that are close together. Such a policy will increase the response time for some requests, probably leading to a larger variation in response time. Although throughput will be higher, some benchmarks constrain the maximum response time to any request, making such optimizations potentially problematic.

In this section, we give some examples of measurements proposed for determining the performance of disk systems. These benchmarks are affected by a variety of system features, including the disk technology, how disks are connected, the memory system, the processor, and the file system provided by the operating system.

Before we discuss these benchmarks, we need to address a confusing point about terminology and units. The performance of I/O systems depends on the rate at which the system transfers data. The transfer rate depends on the clock rate, which is typically given in $\text{GHz} = 10^9$ cycles per second. The transfer rate is usually quoted in GB/sec . In I/O systems, GBs are measured using base 10 (i.e., $1 \text{ GB} = 10^9 = 1,000,000,000$ bytes), unlike main memory where base 2 is used (i.e., $1 \text{ GB} = 2^{30} = 1,073,741,824$). In addition to adding confusion, this difference introduces the need to convert between base 10 ($1\text{K} = 1000$) and base 2 ($1\text{K} = 1024$) because many I/O accesses are for data blocks that have a size that is a power of two. Rather than complicate all our examples by accurately converting one of the two measurements, we make note here of this distinction and the fact that treating the two measures as if the units were identical introduces a small error. We illustrate this error in Section 8.9.

transaction processing A type of application that involves handling small short operations (called transactions) that typically require both I/O and computation. Transaction processing applications typically have both response time requirements and a performance measurement based on the throughput of transactions.

I/O rate Performance measure of I/Os per unit time, such as reads per second.

data rate Performance measure of bytes per unit time, such as GB/second.

Transaction Processing I/O Benchmarks

Transaction processing (TP) applications involve both a response time requirement and a performance measurement based on throughput. Furthermore, most of the I/O accesses are small. Because of this, TP applications are chiefly concerned with **I/O rate**, measured as the number of disk accesses per second, as opposed to **data rate**, measured as bytes of data per second. TP applications generally involve changes to a large database, with the system meeting some response time requirements as well as gracefully handling certain types of failures. These applications are extremely critical and cost-sensitive. For example, banks normally use TP systems because they are concerned about a range of characteristics. These include making sure transactions aren't lost, handling transactions quickly, and minimizing the cost of processing each transaction. Although dependability in the face of failure is an absolute requirement in such systems, both response time and throughput are critical to building cost-effective systems.

A number of transaction processing benchmarks have been developed. The best-known set of benchmarks is a series developed by the Transaction Processing Council (TPC).

TPC-C, initially created in 1992, simulates a complex query environment. TPC-H models ad hoc decision support—the queries are unrelated and knowledge of past queries cannot be used to optimize future queries; the result is that query execution times can be very long. TPC-R simulates a business decision support system where users run a standard set of queries. In TPC-R, preknowledge of the queries is taken for granted, and the DBMS can be optimized to run these queries. TPC-W is a Web-based transaction benchmark that simulates the activities of a business-oriented transactional Web server. It exercises the database system as well as the underlying Web server software. The TPC benchmarks are described at www.tpc.org.

All the TPC benchmarks measure performance in transactions per second. In addition, they include a response time requirement, so that throughput performance is measured only when the response time limit is met. To model real-world systems, higher transaction rates are also associated with larger systems, both in terms of users and the size of the database that the transactions are applied to. Finally, the system cost for a benchmark system must also be included, allowing accurate comparisons of cost-performance.

File System and Web I/O Benchmarks

File systems, which are stored on disks, have a different access pattern. For example, measurements of UNIX file systems in an engineering environment have found that 80% of accesses are to files of less than 10 KB and that 90% of all file accesses are to data with sequential addresses on the disk. Furthermore, 67% of the accesses were reads, 27% were writes, and 6% were read-modify-write accesses, which read data, modify it, and then rewrite the same location. Such

measurements have led to the creation of synthetic file system benchmarks. One of the most popular of such benchmarks has five phases, using 70 files:

- *MakeDir*: Constructs a directory subtree that is identical in structure to the given directory subtree
- *Copy*: Copies every file from the source subtree to the target subtree
- *ScanDir*: Recursively traverses a directory subtree and examines the status of every file in it
- *ReadAll*: Scans every byte of every file in a subtree once
- *Make*: Compiles and links all the files in a subtree

As we will see in Section 8.7, the design of an I/O system involves knowing what the workload is.

In addition to processor benchmarks, SPEC offers both a file server benchmark (SPECFS) and a Web server benchmark (SPECWeb). SPECFS is a benchmark for measuring NFS (Network File System) performance using a script of file server requests; it tests the performance of the I/O system, including both disk and network I/O, as well as the processor. SPECFS is a throughput-oriented benchmark but with important response time requirements. SPECWeb is a Web server benchmark that simulates multiple clients requesting both static and dynamic pages from a server, as well as clients posting data to the server.

I/O Performance versus Processor Performance

Amdahl's law in Chapter 2 reminds us that neglecting I/O is dangerous. A simple example demonstrates this.

Impact of I/O on System Performance

Suppose we have a benchmark that executes in 100 seconds of elapsed time, where 90 seconds is CPU time and the rest is I/O time. If CPU time improves by 50% per year for the next five years but I/O time doesn't improve, how much faster will our program run at the end of five years?

We know that

$$\text{Elapsed time} = \text{CPU time} + \text{I/O time}$$

$$100 = 90 + \text{I/O time}$$

$$\text{I/O time} = 10 \text{ seconds}$$

EXAMPLE

ANSWER

The new CPU times and the resulting elapsed times are computed in the following table:

After <i>n</i> years	CPU time	I/O time	Elapsed time	% I/O time
0	90 seconds	10 seconds	100 seconds	10%
1	$\frac{90}{1.5} = 60$ seconds	10 seconds	70 seconds	14%
2	$\frac{60}{1.5} = 40$ seconds	10 seconds	50 seconds	20%
3	$\frac{40}{1.5} = 27$ seconds	10 seconds	37 seconds	27%
4	$\frac{27}{1.5} = 18$ seconds	10 seconds	28 seconds	36%
5	$\frac{18}{1.5} = 12$ seconds	10 seconds	22 seconds	45%

The improvement in CPU performance over five years is

$$\frac{90}{12} = 7.5$$

However, the improvement in elapsed time is only

$$\frac{100}{22} = 4.5$$

and the I/O time has increased from 10% to 45% of the elapsed time.

Check Yourself

Are the following true or false? Unlike processor benchmarks, I/O benchmarks

- 1. concentrate on throughput rather than latency
- 2. can require that the data set scale in size or number of users to achieve performance milestones
- 3. come from organizations rather than from individuals

8.7 Designing an I/O System

There are two primary types of specifications that designers encounter in I/O systems: latency constraints and bandwidth constraints. In both cases, knowledge of the traffic pattern affects the design and analysis.

Latency constraints involve ensuring that the latency to complete an I/O operation is bounded by a certain amount. In the simple case, the system may be

unloaded, and the designer must ensure that some latency bound is met either because it is critical to the application or because the device must receive certain guaranteed service to prevent errors. Examples of the latter are similar to the analysis we looked at in the previous section. Likewise, determining the latency on an unloaded system is relatively easy, since it involves tracing the path of the I/O operation and summing the individual latencies.

Finding the average latency (or distribution of latency) under a load is a much more complex problem. Such problems are tackled either by queuing theory (when the behavior of the workload requests and I/O service times can be approximated by simple distributions) or by simulation (when the behavior of I/O events is complex). Both topics are beyond the limits of this text.

Designing an I/O system to meet a set of bandwidth constraints given a workload is the other typical problem designers face. Alternatively, the designer may be given a partially configured I/O system and be asked to balance the system to maintain the maximum bandwidth achievable as dictated by the preconfigured portion of the system. This latter design problem is a simplified version of the first.

The general approach to designing such a system is as follows:

1. Find the weakest link in the I/O system, which is the component in the I/O path that will constrain the design. Depending on the workload, this component can be anywhere, including the CPU, the memory system, the backplane bus, the I/O controllers, or the devices. Both the workload and configuration limits may dictate where the weakest link is located.
2. Configure this component to sustain the required bandwidth.
3. Determine the requirements for the rest of the system and configure them to support this bandwidth.

The easiest way to understand this methodology is with an example.

I/O System Design

Consider the following computer system:

- A CPU that sustains 3 billion instructions per second and averages 100,000 instructions in the operating system per I/O operation
- A memory backplane bus capable of sustaining a transfer rate of 1000 MB/sec
- SCSI Ultra320 controllers with a transfer rate of 320 MB/sec and accommodating up to 7 disks
- Disk drives with a read/write bandwidth of 75 MB/sec and an average seek plus rotational latency of 6 ms

EXAMPLE

ANSWER

If the workload consists of 64 KB reads (where the block is sequential on a track) and the user program needs 200,000 instructions per I/O operation, find the maximum sustainable I/O rate and the number of disks and SCSI controllers required. Assume that the reads can always be done on an idle disk if one exists (i.e., ignore disk conflicts).

The two fixed components of the system are the memory bus and the CPU. Let's first find the I/O rate that these two components can sustain and determine which of these is the bottleneck. Each I/O takes 200,000 user instructions and 100,000 OS instructions, so

Maximum I/O rate of CPU =

$$\frac{\text{Instruction execution rate}}{\text{Instructions per I/O}} = \frac{3 \times 10^9}{(200 + 100) \times 10^3} = 10,000 \frac{\text{I/Os}}{\text{second}}$$

Each I/O transfers 64 KB, so

$$\text{Maximum I/O rate of bus} = \frac{\text{Bus bandwidth}}{\text{Bytes per I/O}} = \frac{1000 \times 10^6}{64 \times 10^3} = 15,625 \frac{\text{I/Os}}{\text{second}}$$

The CPU is the bottleneck, so we can now configure the rest of the system to perform at the level dictated by the CPU, 10,000 I/Os per second.

Let's determine how many disks we need to be able to accommodate 10,000 I/Os per second. To find the number of disks, we first find the time per I/O operation at the disk:

$$\begin{aligned} \text{Time per I/O at disk} &= \text{Seek} + \text{rotational time} + \text{Transfer time} \\ &= 6 \text{ ms} + \frac{64 \text{ KB}}{75 \text{ MB/sec}} = 6.9 \text{ ms} \end{aligned}$$

Thus, each disk can complete 1000 ms/6.9 ms or 146 I/Os per second. To saturate the CPU requires 10,000 I/Os per second, or 10,000/146 ^a 69 disks.

To compute the number of SCSI buses, we need to check the average transfer rate per disk to see if we can saturate the bus, which is given by

$$\text{Transfer rate} = \frac{\text{Transfer size}}{\text{Transfer time}} = \frac{64 \text{ KB}}{6.9 \text{ ms}} \text{ a } 9.56 \text{ MB/sec}$$

The maximum number of disks per SCSI bus is 7, which won't saturate this bus. This means we will need 69/7, or 10 SCSI buses and controllers.

Notice the significant number of simplifying assumptions that are needed to do this example. In practice, many of these simplifications might not hold for critical I/O-intensive applications (such as databases). For this reason, simulation is often the only realistic way to predict the I/O performance of a realistic workload.

8.8

Real Stuff: A Digital Camera

Digital cameras are basically embedded computers with removable, writable, non-volatile, storage, and interesting I/O devices. Figure 8.14 shows our example.



FIGURE 8.14 The Sanyo VPC-SX500 with Flash memory card and IBM Microdrive.

Although newer cameras offer more pixels per picture, the principles are the same. This 1360 × 1024 pixel digital camera stores pictures either using CompactFlash memory or using a IBM Microdrive. This photo was taken using a 340 MB microdrive and a 8 MB CompactFlash memory. As Figure 8.15 shows, in 2004 the capacities are as large as 1 GB to 4 GB. It is 4.3 inches wide × 2.5 inches high × 1.6 inches deep, and it weighs 7.4 ounces. In addition to taking a still picture and converting it to JPEG format every 0.9 seconds, it can record a Quick Time video clip at VGA size (640 × 480). One technological advantage is the use of a custom system on a chip to reduce size and power, so the camera only needs two AA batteries to operate versus four in other digital cameras.

When powered on, the microprocessor first runs diagnostics on all components and writes any error messages to the liquid crystal display (LCD) on the back of the camera. This camera uses a 1.8-inch low-temperature polysilicon TFT color LCD. When photographers take pictures, they first hold the shutter halfway so that the microprocessor can take a light reading. The microprocessor then keeps the shutter open to get the necessary light, which is captured by a charged-couple device (CCD) as red, green, and blue pixels.

For the camera in Figure 8.14, the CCD is a 1/2-inch, 1360 × 1024 pixel, progressive-scan chip. The pixels are scanned out row by row and then passed through routines for white balance, color, and aliasing correction, and then stored in a 4 MB frame buffer. The next step is to compress the image into a standard format, such as JPEG, and store it in the removable Flash memory. The photographer picks the compression, in this camera called either fine or normal, with a compression ratio of 10 to 20 times. A fine-quality compressed image takes less than 0.5 MB, and a normal-quality compressed image takes about 0.25 MB. The microprocessor then updates the LCD display to show that there is room for one less picture.

Although the previous paragraph covers the basics of a digital camera, there are many more features that are included: showing the recorded images on the color LCD display; sleep mode to save battery life; monitoring battery energy; buffering to allow recording a rapid sequence of uncompressed images; and, in this camera, video recording using MPEG format and audio recording using WAV format.

This camera allows the photographer to use a Microdrive disk instead of CompactFlash memory. Figure 8.15 compares CompactFlash and the IBM Microdrive.

Characteristics	Sandisk Type I CompactFlash SDCFB-128-768	Sandisk Type II CompactFlash SDCFB-1000-768	Hitachi 4 GB Microdrive DSCM-10340
Formatted data capacity (MB)	128	1000	4000
Bytes per sector	512	512	512
Data transfer rate (MB/sec)	4 (burst)	4 (burst)	4–7
Link speed to buffer (MB/sec)	6	6	33
Power standby/operating (W)	0.15/0.66	0.15/0.66	0.07/0.83
Size: height × width × depth (inches)	1.43 × 1.68 × 0.13	1.43 × 1.68 × 0.13	1.43 × 1.68 × 0.16
Weight in grams (454 grams/pound)	11.4	13.5	16
Write cycles before sector wear-out	300,000	300,000	not applicable
Mean time between failures (hours)	> 1,000,000	> 1,000,000	(see caption)
Best price (2004)	\$40	\$200	\$480

FIGURE 8.15 Characteristics of three storage alternatives for digital cameras. Hitachi matches the Type II form factor in the Microdrive, while the CompactFlash card uses that space to include many more Flash chips. Hitachi does not quote MTTF for the 1.0-inch drives, but the service life is five years or 8800 powered-on hours, whichever is first. They rotate at 3600 RPM and have 12 ms seek times.

The CompactFlash standard package was proposed by Sandisk Corporation in 1994 for the PCMCIA-ATA cards of portable PCs. Because it follows the ATA interface, it simulates a disk interface including seek commands, logical tracks, and so on. It includes a built-in controller to support many types of Flash memory and to help with chip yield for Flash memories by mapping out bad blocks.

The electronic brain of this camera is an embedded computer with several special functions embedded on the chip. Figure 8.16 shows the block diagram of a chip similar to the one in the camera. Such chips have been called *systems on a chip* (SOC) because they essentially integrate into a single chip all the parts that were found on a small printed circuit board of the past. SOC generally reduces size and lowers power compared to less integrated solutions. The manufacturer claims the SOC enables the camera to operate on half the number of batteries and to offer a smaller form factor than competitors' cameras.

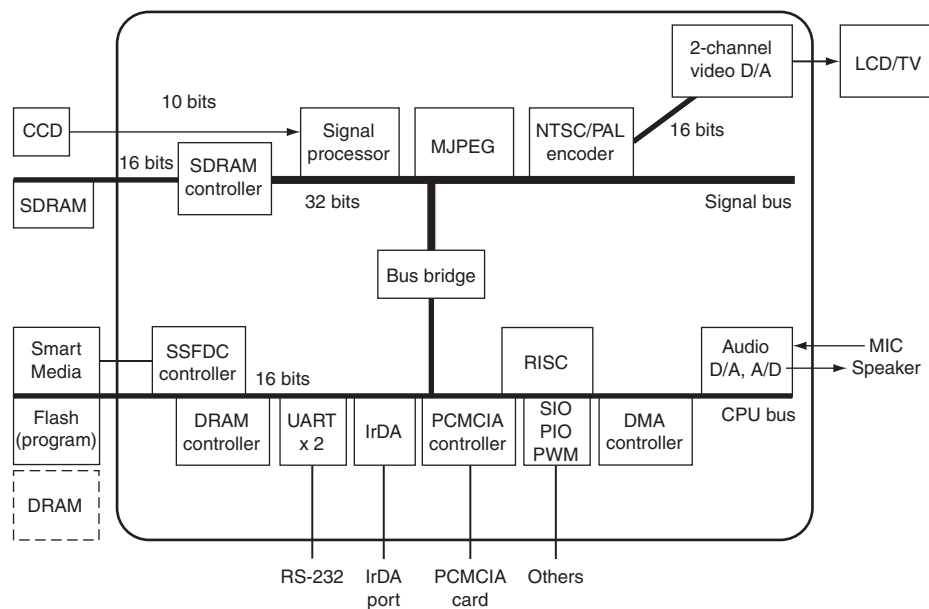


FIGURE 8.16 The system on a chip (SOC) found in Sanyo digital cameras. This block diagram is for the predecessor of the SOC in the camera in Figure 8.14. The successor SOC, called Super Advanced IC, uses three buses instead of two, operates at 60 MHz, consumes 800 mW, and fits 3.1M transistors in a 10.2 × 10.2 mm die using a 0.35-micron process. Note that this embedded system has twice as many transistors as the state-of-the-art, high-performance microprocessor in 1990! The SOC in the figure is limited to processing 1024 × 768 pixels, but its successor supports 1360 × 1024 pixels. (See Okada, Matsuda, Yamada, and Kobayashi [1999]).